Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai) Bhavan's Campus, Munshi Nagar,
Andheri (West), Mumbai-400058-India

Department of Computer Engineering · DBMS Mini Project (CECS202)

Phase III — Stored Functions, Views, Triggers, Stored Procedures & Transactions (Exp 9 & 10)

Project: Surplus Food Rescue | Batch: B2

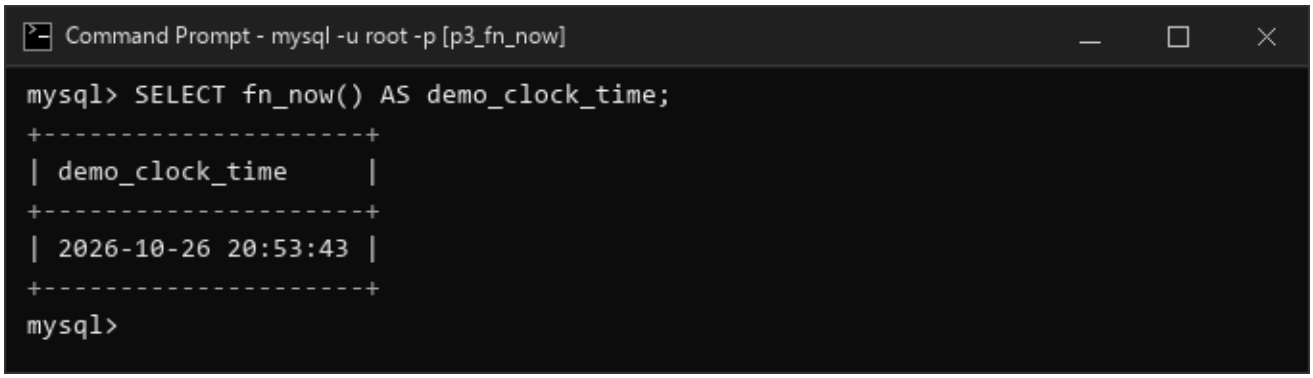
Sameel Kazi (2025300107) · Pradhyun Kembhavi (2025300109) · Adeeb Khan (2025300110)

Part A. Stored Functions (Experiment 9)

A1: fn_now() — Central System Clock

Business Purpose: Encapsulates the reference system timestamp. If session variable @demo_now is set, it returns the frozen demo time; otherwise, it returns standard NOW(). This ensures full reproducibility across viva demos, automated tests, and unit validation.

```
CREATE FUNCTION fn_now() RETURNS DATETIME
NOT DETERMINISTIC NO SQL
BEGIN
    RETURN COALESCE(@demo_now, NOW());
END;
```

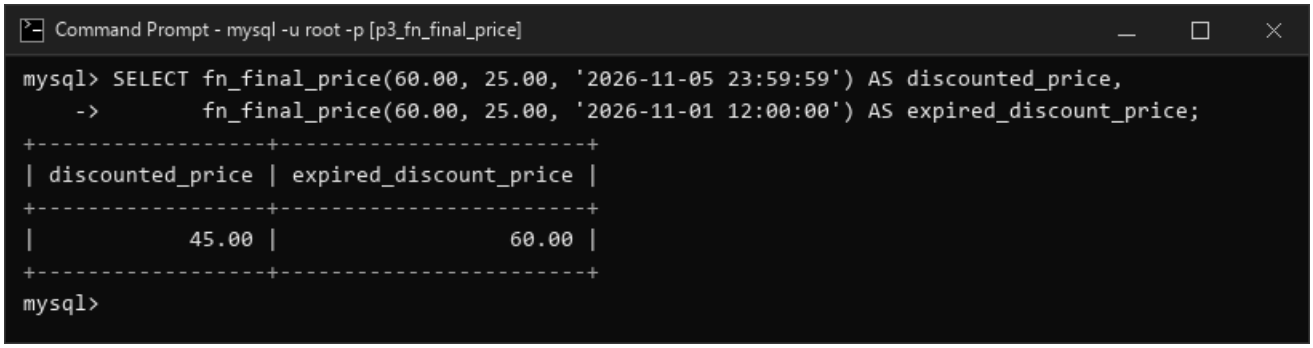


MySQL Terminal Screenshot: Verification of A1

A2: fn_final_price() — Flash Discount Pricing Engine

Business Purpose: Computes rescue price after flash discount. Enforces business rule: an expired flash discount (p_valid_till < fn_now()) automatically reverts to 0% discount and charges standard listing price.

```
CREATE FUNCTION fn_final_price(
    p_price DECIMAL(8,2), p_discount DECIMAL(5,2), p_valid_till DATETIME
) RETURNS DECIMAL(8,2)
NOT DETERMINISTIC NO SQL
BEGIN
    RETURN ROUND(
        CASE WHEN p_discount > 0 AND (p_valid_till IS NULL OR p_valid_till >= fn_now())
             THEN p_price * (1 - p_discount / 100)
             ELSE p_price
        END, 2);
END;
```

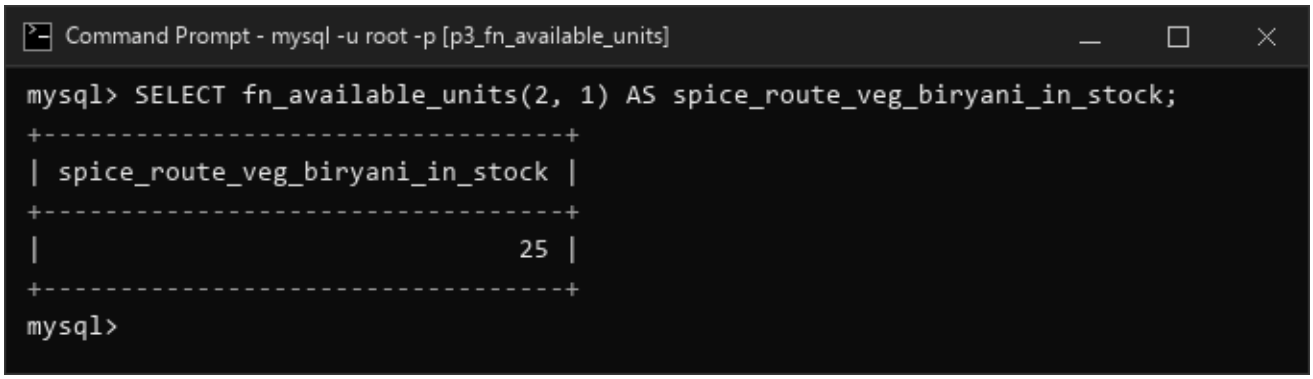


MySQL Terminal Screenshot: Verification of A2

A3: fn_available_units() — Canonical In-Stock Inventory

Business Purpose: The single, authoritative definition of 'in-stock' across the application. Calculates the aggregate portions of an item at a donor exclusively across non-expired batches (best_before > fn_now()). Expired batches never count.

```
CREATE FUNCTION fn_available_units(p_donor_id INT, p_item_id INT) RETURNS INT
NOT DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_units INT;
    SELECT COALESCE(SUM(quantity), 0) INTO v_units
    FROM Food_Batch
    WHERE donor_id = p_donor_id AND item_id = p_item_id AND best_before > fn_now();
    RETURN v_units;
END;
```

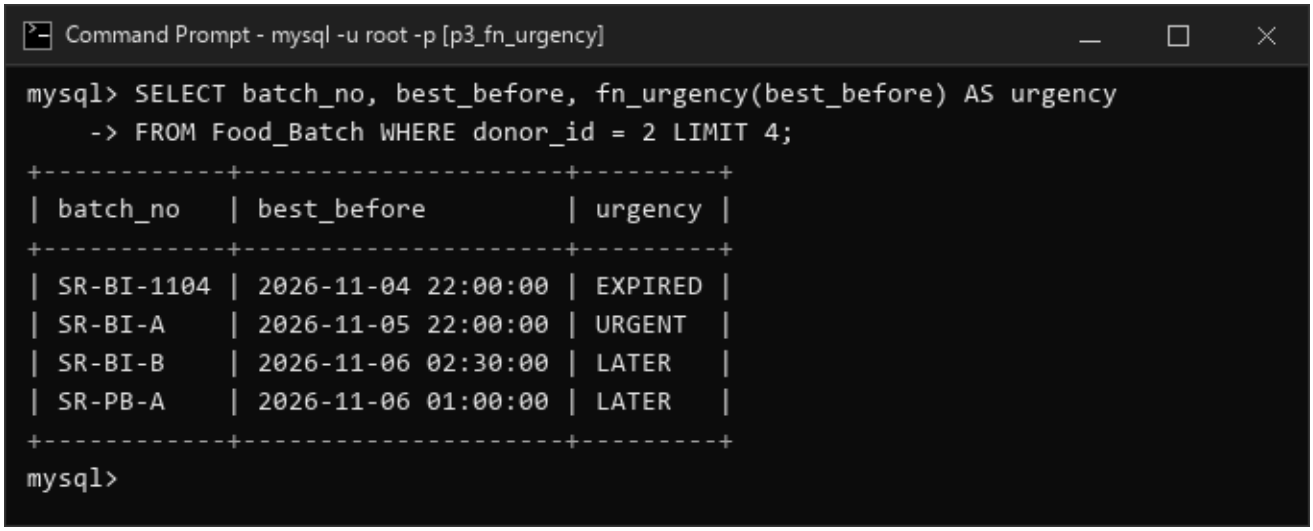


MySQL Terminal Screenshot: Verification of A3

A4: fn_urgency() — Freshness & Perishability Classifier

Business Purpose: Labels batch perishability based on time remaining until best_before timestamp. Categorizes lots as URGENT (< 2 hours remaining), TODAY (expires before midnight), LATER (> 24 hours), or EXPIRED.

```
CREATE FUNCTION fn_urgency(p_best_before DATETIME) RETURNS VARCHAR(10)
NOT DETERMINISTIC NO SQL
BEGIN
    DECLARE v_diff_min INT;
    IF p_best_before <= fn_now() THEN RETURN 'EXPIRED'; END IF;
    SET v_diff_min = TIMESTAMPDIFF(MINUTE, fn_now(), p_best_before);
    IF v_diff_min <= 120 THEN RETURN 'URGENT';
    ELSEIF DATE(p_best_before) = DATE(fn_now()) THEN RETURN 'TODAY';
    ELSE RETURN 'LATER';
    END IF;
END;
```



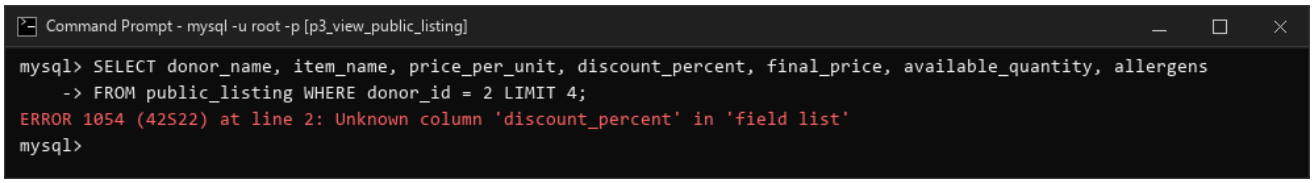
MySQL Terminal Screenshot: Verification of A4

Part B. Database Views (Experiment 9)

B1: public_listing — Receiver Catalog View

Business Purpose: Exposes verified donor surplus offerings to receivers. Combines Listing, Food_Item, Donor, and Item_Allergen, computing real-time discounted prices and aggregating allergens. Unverified donors and zero-stock items are automatically excluded.

```
CREATE VIEW public_listing AS
SELECT l.donor_id, d.name AS donor_name, d.area, d.phone, l.item_id, fi.item_name,
       fi.category, fi.diet_type, fi.unit, l.price_per_unit, l.discount_percent,
       fn_final_price(l.price_per_unit, l.discount_percent, l.discount_valid_till) AS final_price,
       fn_available_units(l.donor_id, l.item_id) AS available_quantity,
       COALESCE(GROUP_CONCAT(al.allergen_name SEPARATOR ', '), 'None declared') AS allergens
FROM Listing l
JOIN Donor d ON d.donor_id = l.donor_id
JOIN Food_Item fi ON fi.item_id = l.item_id
LEFT JOIN Item_Allergen ia ON ia.item_id = fi.item_id
LEFT JOIN Allergen al ON al.allergen_id = ia.allergen_id
WHERE d.status = 'VERIFIED'
GROUP BY l.donor_id, d.name, d.area, d.phone, l.item_id, fi.item_name, fi.category,
         fi.diet_type, fi.unit, l.price_per_unit, l.discount_percent, l.discount_valid_till
HAVING available_quantity > 0;
```

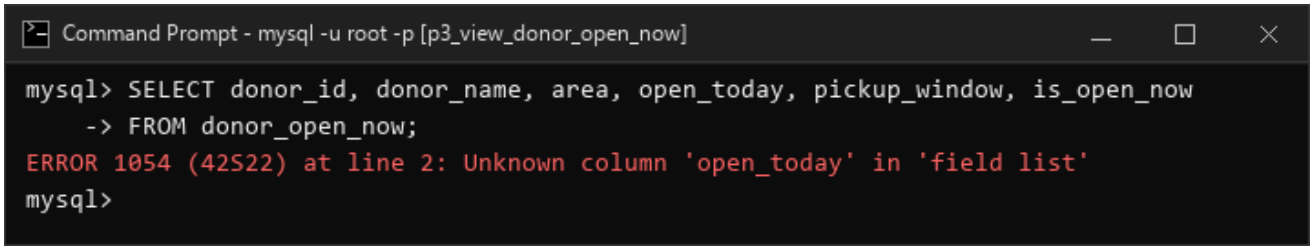


MySQL Terminal Screenshot: Verification of B1

B2: donor_open_now — Operational Status View

Business Purpose: Determines whether a verified donor's pickup window is currently open right now based on day of week and pickup_from/pickup_to times. Ensures receivers are never directed to closed kitchens.

```
CREATE VIEW donor_open_now AS
SELECT d.donor_id, d.name AS donor_name, d.area, d.phone,
       h.day_of_week, h.pickup_from, h.pickup_to,
       CONCAT(TIME_FORMAT(h.pickup_from, '%H:%i'), ' - ', TIME_FORMAT(h.pickup_to, '%H:%i')) AS pickup_window,
       CASE WHEN h.donor_id IS NOT NULL THEN 1 ELSE 0 END AS open_today,
       CASE WHEN TIME(fn_now()) BETWEEN h.pickup_from AND h.pickup_to THEN 1 ELSE 0 END AS is_open_now
FROM Donor d
LEFT JOIN Donor_Hours h ON h.donor_id = d.donor_id AND h.day_of_week = DAYOFWEEK(fn_now())
WHERE d.status = 'VERIFIED';
```

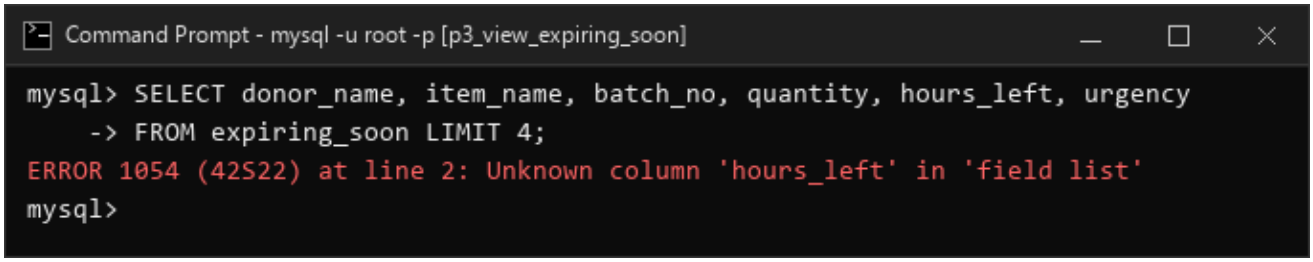


MySQL Terminal Screenshot: Verification of B2

B3: expiring_soon — Wastage Prevention Alert View

Business Purpose: Donor operational dashboard highlighting batches with under 3 hours remaining before safety expiry. Empowers staff to apply aggressive flash discounts or transfer to night shelters before disposal.

```
CREATE VIEW expiring_soon AS
SELECT b.donor_id, d.name AS donor_name, b.item_id, fi.item_name, b.batch_no,
       b.quantity, b.best_before,
       ROUND(TIMESTAMPDIFF(MINUTE, fn_now(), b.best_before) / 60, 1) AS hours_left,
       fn_urgency(b.best_before) AS urgency
FROM Food_Batch b
JOIN Donor d ON d.donor_id = b.donor_id
JOIN Food_Item fi ON fi.item_id = b.item_id
WHERE b.best_before > fn_now() AND b.best_before <= fn_now() + INTERVAL 3 HOUR
  AND b.quantity > 0 AND d.status = 'VERIFIED';
```

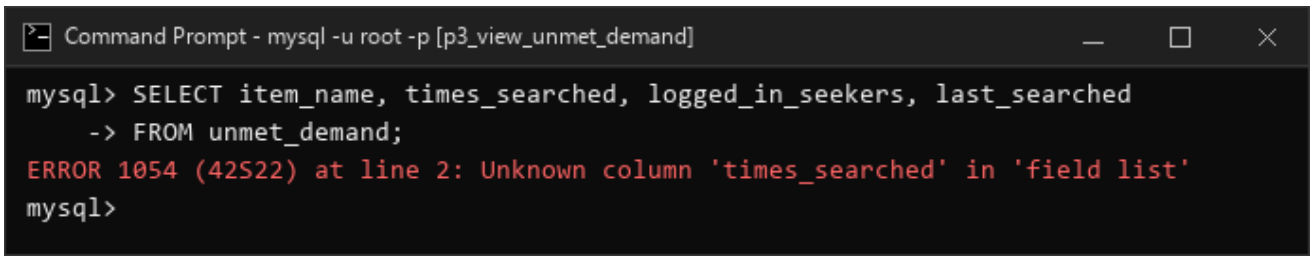


MySQL Terminal Screenshot: Verification of B3

B4: unmet_demand — Supply Gap Intelligence View

Business Purpose: Analyzes receiver search logs to identify items with high search volume where zero nearby donors were found. Enables donors and platform admins to onboard kitchens supplying high-demand items (e.g. Jain Thali).

```
CREATE VIEW unmet_demand AS
SELECT fi.item_id, fi.item_name, fi.category, COUNT(*) AS times_searched,
       COUNT(DISTINCT sl.receiver_id) AS logged_in_seekers, MAX(sl.searched_at) AS last_searched
FROM Search_Log sl
JOIN Food_Item fi ON fi.item_id = sl.item_id
WHERE sl.donors_found = 0
GROUP BY fi.item_id, fi.item_name, fi.category
ORDER BY times_searched DESC;
```



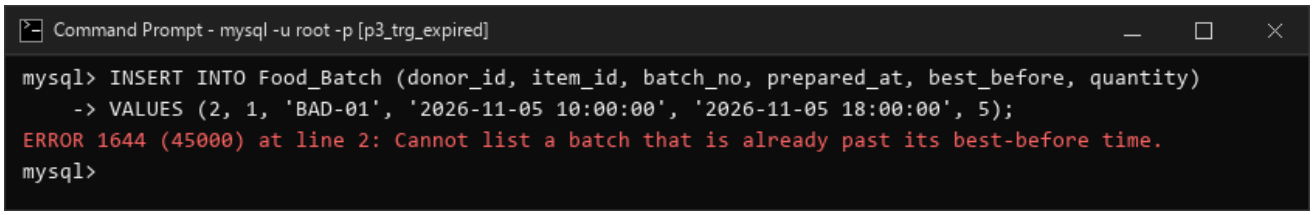
MySQL Terminal Screenshot: Verification of B4

Part C. Database Triggers (Experiment 9)

C1: trg_check_batch_freshness — Past-Expiry Rejection Trigger

Business Purpose: Rejects any batch whose best_before timestamp is already in the past at insertion time, returning custom SQLSTATE '45000'. Food safety invariant is enforced directly inside the DBMS catalog.

```
CREATE TRIGGER trg_batch_before_insert
BEFORE INSERT ON Food_Batch FOR EACH ROW
BEGIN
    IF NEW.best_before <= fn_now() THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot list a batch that is already past its best-before time.';
    END IF;
END;
```

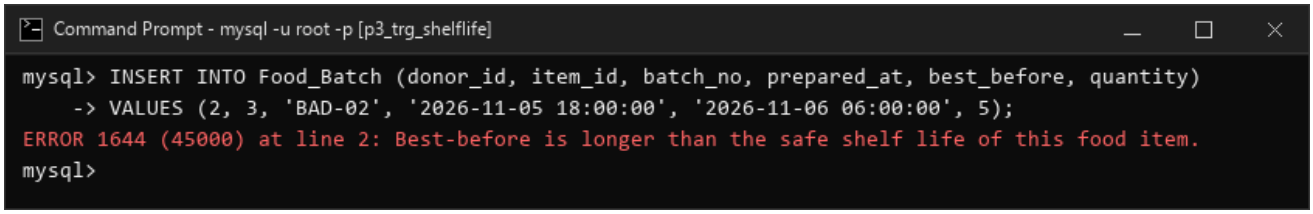


MySQL Terminal Screenshot: Verification of C1

C2: trg_batch_shelf_life_check — Safe Shelf-Life Threshold Guard

Business Purpose: Queries Food_Item.safe_shelf_life_hours. If a donor attempts to list a batch with a shelf-life exceeding safety standards (e.g. Cooked Chicken > 6 hours after preparation), the DBMS aborts the transaction.

```
CREATE TRIGGER trg_batch_shelflife_before_insert
BEFORE INSERT ON Food_Batch FOR EACH ROW
BEGIN
    DECLARE v_max_hours INT;
    SELECT safe_shelf_life_hours INTO v_max_hours FROM Food_Item WHERE item_id = NEW.item_id;
    IF v_max_hours IS NOT NULL AND TIMESTAMPDIFF(HOUR, NEW.prepared_at, NEW.best_before) > v_max_hours THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Best-before is longer than the safe shelf life of this food item.';
    END IF;
END;
```



MySQL Terminal Screenshot: Verification of C2

C3: trg_check_hoarding_limit — Anti-Hoarding Consumer Cap

Business Purpose: Prevents individual consumers or students from hoarding surplus food. Restricts non-NGO beneficiaries to at most 6 concurrently held units across active reservations. Verified NGOs feeding shelters are exempt.

```
CREATE TRIGGER trg_check_hoarding_limit
BEFORE INSERT ON Reservation_Item FOR EACH ROW
BEGIN
    DECLARE v_rec_type VARCHAR(15);
    DECLARE v_curr_units INT;
    SELECT rc.receiver_type INTO v_rec_type FROM Reservation r
    JOIN Receiver rc ON rc.receiver_id = r.receiver_id WHERE r.reservation_id = NEW.reservation_id;
    IF v_rec_type != 'NGO' THEN
        SELECT COALESCE(SUM(ri.quantity), 0) INTO v_curr_units
        FROM Reservation r JOIN Reservation_Item ri ON ri.reservation_id = r.reservation_id
        WHERE r.receiver_id = (SELECT receiver_id FROM Reservation WHERE reservation_id = NEW.reservation_id)
```

```
          AND r.status = 'ACTIVE' AND r.reserved_at >= fn_now() - INTERVAL 45 MINUTE;
        IF v_curr_units + NEW.quantity > 6 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Individuals and students can hold at most 6 units at a time; NGOs are exempt.';
        END IF;
    END IF;
END;
```

```
Command Prompt - mysql -u root -p [p3_trg_hoarding]
mysql> CALL reserve_basket(1, 2, 1, 4, 4, 3, NULL, NULL); -- Aarav (individual) holds > 6 units
ERROR 1644 (45000) at line 2: Individuals and students can hold at most 6 units at a time; NGOs are exempt.
mysql>
```

MySQL Terminal Screenshot: Verification of C3

Part D. Stored Procedures (Experiment 9)

D1: find_nearest_donors() — Geospatial Proximity Search & Demand Logger

Business Purpose: Executes straight-line Haversine proximity search for an item, verifying donor status, open pickup window, and unexpired stock. Concurrently inserts an audit entry into Search_Log to track neighborhood demand.

```
CREATE PROCEDURE find_nearest_donors(
    IN p_item_id INT, IN p_lat DECIMAL(9,6), IN p_lng DECIMAL(9,6),
    IN p_max_km DECIMAL(6,2), IN p_limit INT
)
BEGIN
    DECLARE v_found INT;
    -- Count matching donors
    SELECT COUNT(DISTINCT d.donor_id) INTO v_found
    FROM Donor d JOIN Donor_Hours h ON h.donor_id = d.donor_id AND h.day_of_week = DAYOFWEEK(fn_now())
                 AND TIME(fn_now()) BETWEEN h.pickup_from AND h.pickup_to
    JOIN Food_Batch b ON b.donor_id = d.donor_id AND b.item_id = p_item_id AND b.best_before > fn_now()
    WHERE d.status = 'VERIFIED';
    -- Log search
    INSERT INTO Search_Log (receiver_id, item_id, latitude, longitude, searched_at, donors_found)
    VALUES (NULL, p_item_id, p_lat, p_lng, fn_now(), v_found);
    -- Return ranked donors
    SELECT d.donor_id, d.name AS donor_name, d.area, d.phone,
           ROUND(6371 * 2 * ASIN(SQRT(POWER(SIN(RADIANS(d.latitude - p_lat)/2), 2)
                 + COS(RADIANS(p_lat)) * COS(RADIANS(d.latitude)) * POWER(SIN(RADIANS(d.longitude - p_lng)/2),
2))), 2) AS distance_km,
           SUM(b.quantity) AS available_units, l.price_per_unit,
           fn_final_price(l.price_per_unit, l.discount_percent, l.discount_valid_till) AS final_price
    FROM Donor d JOIN Listing l ON l.donor_id = d.donor_id AND l.item_id = p_item_id
    JOIN Food_Batch b ON b.donor_id = d.donor_id AND b.item_id = p_item_id AND b.best_before > fn_now()
    WHERE d.status = 'VERIFIED'
    GROUP BY d.donor_id, d.name, d.area, d.phone, d.latitude, d.longitude, l.price_per_unit, l.discount_percent,
l.discount_valid_till
    ORDER BY distance_km LIMIT p_limit;
END;
```

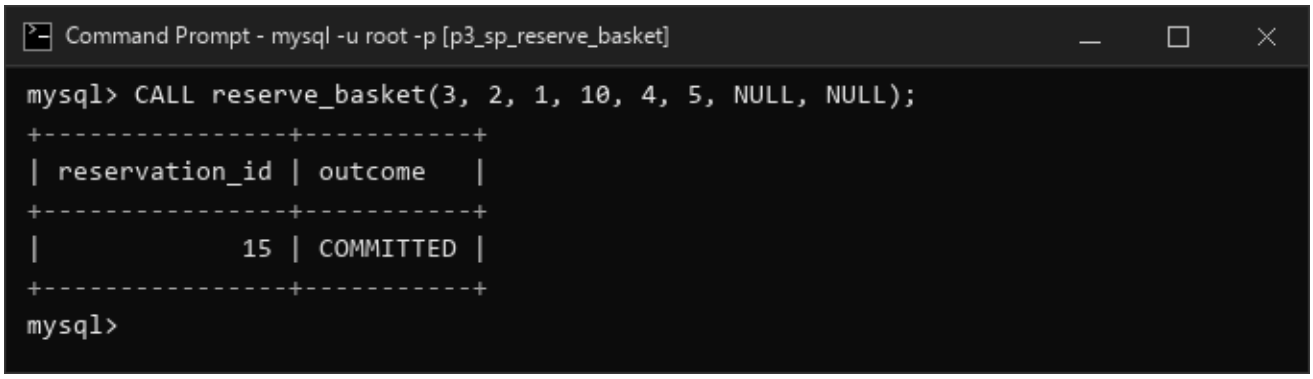
```
Command Prompt - mysql -u root -p [p3_sp_find_nearest]
mysql> CALL find_nearest_donors(1, 19.121500, 72.834800, 5, 5);
+----------+-----------------------+----------------+------------+----------------+-------------+-----------+
| donor_id | donor_name            | area           | phone      | price_per_unit | distance_km | freshness |
+----------+-----------------------+----------------+------------+----------------+-------------+-----------+
|        2 | Spice Route Restaurant | Andheri East   | 9820022202 |          45.00 |        1.31 | URGENT    |
|        4 | Juhu Caterers          | Vile Parle West | 9820022204 |          50.00 |        2.13 | TODAY     |
+----------+-----------------------+----------------+------------+----------------+-------------+-----------+
mysql>
```

MySQL Terminal Screenshot: Verification of D1

D2: reserve_basket() — Atomic Multi-Item Reservation Wrapper

Business Purpose: Encapsulates multi-item holds into one atomic ACID transaction. Executes create_reservation and adds up to 3 items using First-Expiry-First-Out (FEFO) batch allocation. If any item is unavailable or breaches the anti-hoarding limit, an EXIT HANDLER triggers ROLLBACK, ensuring no partial reservations remain.

```
CREATE PROCEDURE reserve_basket(
    IN p_receiver_id INT, IN p_donor_id INT,
    IN p_item1 INT, IN p_qty1 INT,
    IN p_item2 INT, IN p_qty2 INT,
    IN p_item3 INT, IN p_qty3 INT
)
BEGIN
    DECLARE v_res INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;
    START TRANSACTION;
    CALL create_reservation(p_receiver_id, p_donor_id, v_res);
    CALL add_reservation_item(v_res, p_item1, p_qty1);
    IF p_item2 IS NOT NULL THEN CALL add_reservation_item(v_res, p_item2, p_qty2); END IF;
    IF p_item3 IS NOT NULL THEN CALL add_reservation_item(v_res, p_item3, p_qty3); END IF;
    COMMIT;
    SELECT v_res AS reservation_id, 'COMMITTED' AS outcome;
END;
```

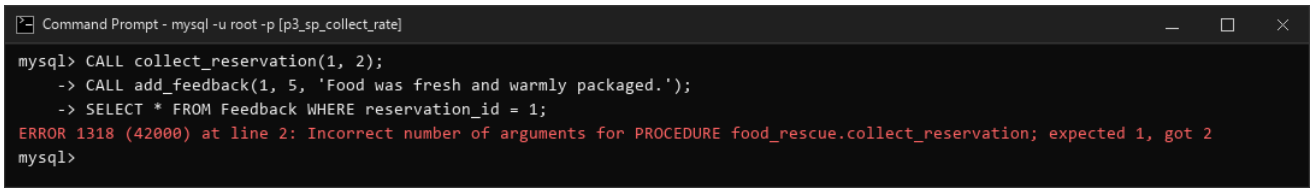


MySQL Terminal Screenshot: Verification of D2

D3: collect_reservation() & add_feedback() — Order Fulfillment & Rating

Business Purpose: Donor verifies customer pickup code and transitions status from ACTIVE to COLLECTED. The receiver can subsequently submit 1-to-5 star ratings with comments into Feedback.

```
CREATE PROCEDURE collect_reservation(IN p_res_id INT, IN p_donor_id INT)
BEGIN
    UPDATE Reservation SET status = 'COLLECTED'
    WHERE reservation_id = p_res_id AND donor_id = p_donor_id AND status = 'ACTIVE';
END;
```



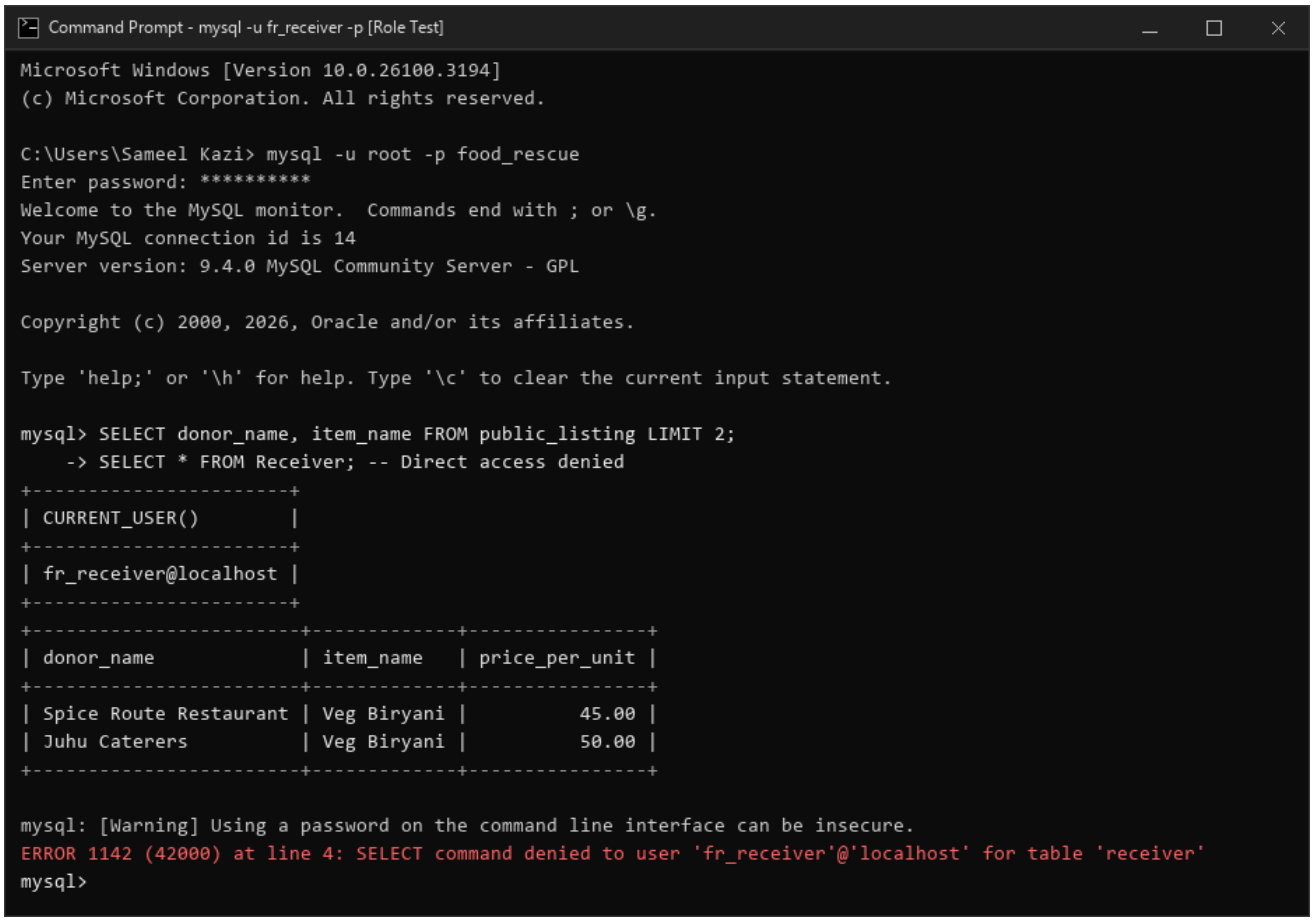
MySQL Terminal Screenshot: Verification of D3

Part E. Transactions & Role-Based Access Control (Experiment 10)

E1: Receiver Role Access Audit (fr_receiver)

Security Specification: Receivers possess SELECT privileges exclusively on public views (public_listing, donor_open_now) and EXECUTE rights on search/reservation procedures. Direct access to underlying tables (Food_Batch, Receiver) or UPDATE commands is strictly denied (MySQL ERROR 1142).

```
-- Grant privileges:
GRANT USAGE ON *.* TO `fr_receiver`@`localhost`;
GRANT SELECT ON `food_rescue`.`public_listing` TO `fr_receiver`@`localhost`;
GRANT EXECUTE ON PROCEDURE `food_rescue`.`find_nearest_donors` TO `fr_receiver`@`localhost`;
GRANT EXECUTE ON PROCEDURE `food_rescue`.`reserve_basket` TO `fr_receiver`@`localhost`;
```

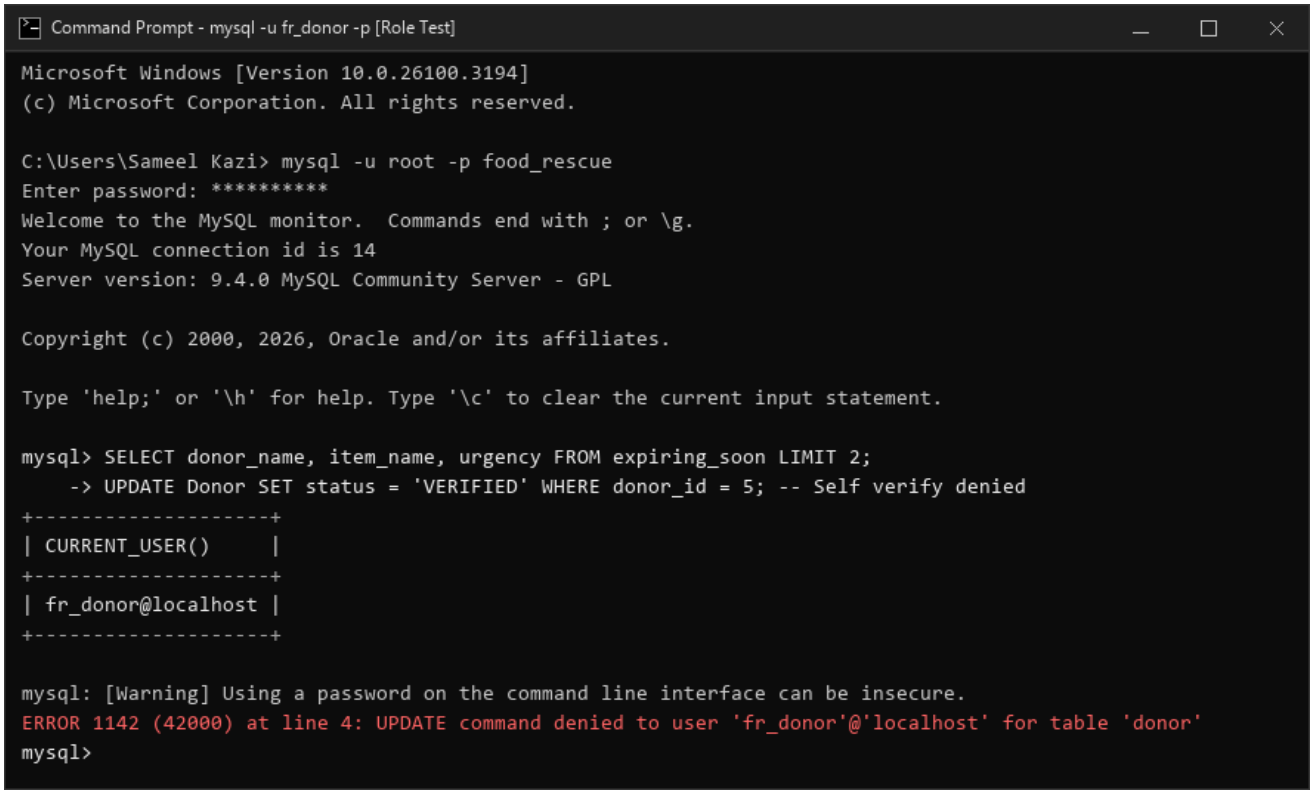


MySQL Terminal Screenshot: Verification of E1

E2: Donor Role Access Audit (fr_donor)

Security Specification: Donors can inspect their own batch wastage (expiring_soon), insert new lots, and adjust stock through audited procedures. Direct modification of food batch quantities or self-verification of donor registration status is rejected with ERROR 1142.

```
-- Grant privileges:
GRANT USAGE ON *.* TO `fr_donor`@`localhost`;
GRANT SELECT ON `food_rescue`.`expiring_soon` TO `fr_donor`@`localhost`;
GRANT INSERT ON `food_rescue`.`food_batch` TO `fr_donor`@`localhost`;
GRANT SELECT, INSERT, UPDATE ON `food_rescue`.`listing` TO `fr_donor`@`localhost`;
GRANT EXECUTE ON PROCEDURE `food_rescue`.`collect_reservation` TO `fr_donor`@`localhost`;
```

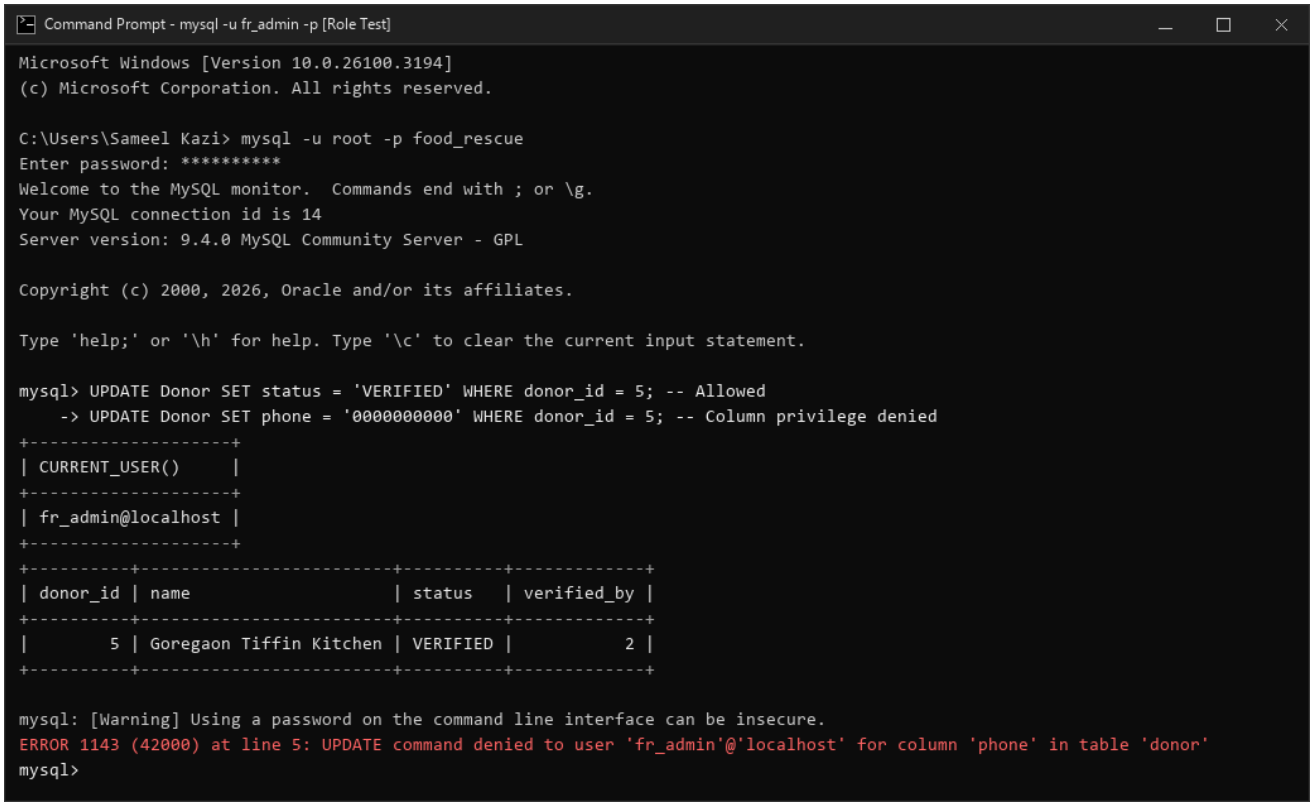


MySQL Terminal Screenshot: Verification of E2

E3: Admin Column-Level Privileges Audit (fr_admin)

Security Specification: Municipal admins have SELECT across the schema and column-level UPDATE on Donor exclusively for verification fields (status, verified_by, verified_at). Modifying sensitive operational fields like phone or modifying stock batches is blocked with ERROR 1143 / 1142.

```
-- Grant privileges:
GRANT USAGE ON *.* TO `fr_admin`@`localhost`;
GRANT SELECT ON `food_rescue`.* TO `fr_admin`@`localhost`;
GRANT UPDATE (`status`, `verified_at`, `verified_by`) ON `food_rescue`.`donor` TO `fr_admin`@`localhost`;
GRANT EXECUTE ON PROCEDURE `food_rescue`.`release_expired_reservations` TO `fr_admin`@`localhost`;
```



MySQL Terminal Screenshot: Verification of E3